ATC Full User Guide

Advanced Trigonometry Calculator Full User Guide

Version: ATC 2.1.9 Language: English Author: Renato Alexandre dos Santos Freitas

This guide is the expanded repository user guide for Advanced Trigonometry Calculator (ATC). It is based on the existing online user guide, the 2.1.8 documentation and subsequent 2.1.9 updates, and the automated regression coverage available in this repository.

1. What ATC is

ATC is a free, open-source Windows command-line mathematical application. It is designed for fast local mathematical work through typed commands and expressions.

Typical use cases:

- quick scientific calculations;
- trigonometry and inverse trigonometry;
- complex-number calculations;
- matrix work;
- statistics and probability;
- DSP helpers;
- polynomial simplification;
- equation solving and systems of equations;
- reusable variables and expressions;
- guided interactive calculation modules;
- TXT-file based command workflows.

2. What ATC is not

ATC deliberately focuses on practical command-driven mathematical computation instead of trying to be a universal CAS.

- ATC is not a full general-purpose CAS like Mathematica, Maple, SageMath, or SymPy.
- ATC is not designed to symbolically prove arbitrary mathematical theorems.
- ATC is not intended to replace domain-specific professional engineering validation tools.
- ATC focuses on numeric computation, command-driven workflows, educational use, implemented symbolic/numeric hybrid features, and fast local execution.
- Some symbolic behavior exists, but only where explicitly supported and tested.

3. Quick Start

ATC is a free, open-source Windows command-line mathematical application. It evaluates typed mathematical expressions, documented commands and guided workflows. It is best understood as a practical local calculator, not as a universal symbolic CAS.

Open ATC by launching `atc.exe` from the built Release folder or through the installed ATC shortcut/launcher when available. The prompt accepts one command or expression per line.

First commands to try:

```
2+2
sin(pi/2)
mode
solve equation(x^2-5*x+6)
solver(x+2)
create matrix(foo\2\2\3)
see variables
exit
```

Basic reading rules:

- _ is used by ATC as a negative marker in many documented command forms and outputs, for example _5 means -5.

- #0, #1, and later indexes refer to previous results in the current session.

- A direct expression is evaluated immediately, such as 2+2.

- A command starts a named ATC action, such as mode or solve equation(...).

- A guided module opens an interactive workflow, such as `financial calculations or unit conversions`.

Recommended first workflow:

```
mode
2+2
sin(pi/2)
solve equation(x^2-5*x+6)
verbose resolution(1)
sin(pi/6)
verbose resolution(0)
```

If a result matters, confirm the angle mode and precision mode before relying on it.

4. Input basics

ATC expressions are typed directly at the prompt.

```
2+3*4
sqrt(9)
sin(pi/2)
log(100)
```

Stable example outputs:

```
#0=14
#1=3
#2=1
#3=2
```

ATC uses _ as a negative marker in many documented inputs and outputs:

```
_6+2
```

Expected result:

```
#0=-4
```

Scientific notation uses uppercase E:

```
1E3
1E-3
```

5. Interactive prompt

ATC 2.1.9 includes a custom prompt editor:

- Tab completes documented commands, mathematical functions, aliases and user functions;

- repeated Tab cycles possible completions;

- `Up` and `Down` navigate command history;

- `Left`, `Right`, `Home`, `End`, `Delete` and `Backspace` edit the current

input line.

This improves speed while keeping calculation semantics in the normal ATC parser.

6. Angle mode

Use:

```
mode
```

The menu offers:

```
radian -> 1
degree -> 2
gradian -> 3
```

Examples:

```
sin(pi/2)
sin(30)
```

Use `sin(pi/2)` in radian mode and `sin(30)` in degree mode.

7. Precision

ATC 2.1.9 supports:

- `double`

- Boost `mp_float`

Persisted precision mode:

```
higherprecision(1)
higherprecision(0)
```

The setting is applied after restarting ATC.

Useful precision commands:

```
dp15dppi
dp50dppi
dp50dpe
dp50dpmaxprecpi
```

Stable high-precision examples:

```
dp50dppi
3.14159265358979323846264338327950288419716939937511

dp50dpe
2.71828182845904523536028747135266249775724709369996
```

`maxprec` is useful when a single expression needs higher precision without changing the persisted precision mode.

8. Constants and previous answers

Common constants:

```
pi
e
i
INF
_INF
true
false
```

Previous result references use ATC result indexes, for example:

```
#0
#1
```

Use this to build calculations step by step.

9. Arithmetic

Examples:

```
2+3*4
(2+3)*4
8/4
2^10
5!
abs(_7)
100rest(3)
100quotient(3)
rtD4D(16)
```

Stable expected outputs include:

```
2+3*4      -> 14
(2+3)*4    -> 20
2^10       -> 1024
5!         -> 120
abs(_7)    -> 7
rtD4D(16)  -> 2
```

10. Trigonometry

Examples:

```
sin(pi/2)
cos(0)
tan(pi/4)
sec(0)
cosec(pi/2)
cotan(pi/4)
asin(1)
acos(1)
atan(1)
acosec(1)
asec(1)
acotan(1)
```

When using degree mode:

```
mode
sin(30)
```

Expected:

```
#n=0.5
```

11. Hyperbolic functions

Examples:

```
sinh(0)
cosh(0)
tanh(0)
sech(0)
cosech(1)
cotanh(1)
asinh(0)
acosh(1)
atanh(0)
asech(1)
acosech(1)
acotanh(2)
```

These are useful for scientific and engineering formulas involving hyperbolic relationships.

12. Logarithms

Examples:

```
log(100)
ln(e)
logb2b(8)
```

Expected:

```
log(100)  -> 2
ln(e)     -> 1
logb2b(8) -> 3
```

13. Complex numbers

ATC supports complex values with `i`.

Examples:

```
2i
3(2i)
sin(30+30i)
asin(sin(30+30i))
```

Complex-number behavior depends on the selected angle mode for trigonometric functions.

14. Automatic multiplication

ATC can infer multiplication in documented forms.

Examples:

```
2pi
pi2
2e
2(3+4)
(1+1)(2+3)
2sin(pi/2)
sin(pi/2)2
2i
3(2i)
```

With variables:

```
x=4
2x
x(4)
(4)x
xpi
pix
xe
xsin(pi/2)
```

When a variable name exists exactly, ATC should prefer the existing variable over splitting it into implied products.

15. Variables

Examples:

```
data=5
data+1
energy=10
energy+1
```

ATC 2.1.9 accepts broader variable names than older parser versions while preserving reserved function and constant names.

Useful commands:

```
see variables
renamed variables
eliminate variables
see results
eliminate results
```

16. Matrices

Create a matrix:

```
create matrix(foo\2\2\3)
```

Matrix helpers:

```
min(foo)
max(foo)
avg(foo)
linsnum(foo)
colsnum(foo)
getlins(foo)
getcols(foo)
```

Matrix arithmetic:

```
foo+foo
foo-foo
foo*foo
foo^2
foo^T
foo^R
foo/2
2*foo
```

Use matrices for tabular numeric workflows, linear algebra helpers and repeated numeric data.

17. Statistics and probability

List helpers:

```
min(3\_1\2)
max(3\_1\2)
avg(2\4\6)
```

Expected:

```
min(3\_1\2) -> -1
max(3\_1\2) -> 3
avg(2\4\6)  -> 4
```

Gaussian/probability helpers:

```
gerror(0)
gerrorc(0)
gerrorinv(0)
gerrorcinv(1)
qfunc(0)
qfuncinv(0.5)
```

Guided module:

```
statistics calculations
```

Use the guided module for menu-based workflows such as variance and standard deviation where available.

18. DSP

Signal helpers:

```
sinc(0)
sinc(1)
fft(1\0\0\0)
fft(1\2\3)
ifft(1\0\0\0)
ifft(1\1\1\1)
```

Use these commands for compact signal-processing experiments and educational checks.

19. Polynomial tools

Simplify supported polynomial forms:

```
simplify polynomial(x)
simplify polynomial(x+1)
simplify polynomial(x+x+1)
simplify polynomial(x^2-5*x+6)
simplify polynomial((x-2)*(x-3))
simplify polynomial((x+1)*(x+1))
simplify polynomial((x-1)^2)
simplify polynomial((x+2)*(x-2))
simplify polynomial((x+2i)*(x-2i))
simplify polynomial(((x-5)(x+2))/(x-5))
```

Convert roots to a polynomial:

```
roots to polynomial(2\3)
roots to polynomial(2\7\12)
roots to polynomial(6.5i\_6.5i)
roots to polynomial(1+2i\1-2i)
roots to polynomial(0\1\2)
```

Stable example:

```
roots to polynomial(2\3)
(1+0i)x^2+(_5+0i)x^1+(6+0i)
```

20. Equation solving

Quadratic command:

```
solve quadratic equation(1\_5\6)
solve quadratic equation(1\_2\1)
solve quadratic equation(1\2\5)
```

Systems:

```
solve equations system(1\1\5;1\_1\1)
```

Polynomial equations:

```
solve equation(x+2)
solve equation(x^2-5*x+6)
solve equation(x^7-12)
solve equation(1\2\3\4\5)
solve equation((x-1)(x-2))
solve equation(((x-5)(x+2))/(x-5))
solve equation(x^25-100)
```

Stable examples:

```
solve equation(x+2)
x1=-2

solve equation(x^2-5*x+6)
x1=3
x2=2
```

21. Solver

The `solver(...)` command supports numerical solving paths and selected fast-path normalizations. ATC 2.1.9 validates each candidate against the original expression before returning it. A solution may be real or complex. If the earlier solver path does not produce a valid root, a general numerical fallback is attempted.

Examples:

```
solver(sin(x)-0.5)
solver(0.5-sin(x))
solver(cos(x)-cos(60))
solver(tan(x)-tan(45))
solver(asin(sin(30))-x)
solver(qfunc(x)-qfunc(0.34233))
solver(x^2-12x-9)
solver(x+2)
solver(((x-5)(x+2))/(x-5))
solver((x-e+pii)(x-e-pii))
solver(x^2-2)
solver(x^2+1)
solver(x^2-2i)
solver(x^2+22)
```

The last four examples cover validated real and complex polynomial roots. When no candidate satisfies the original expression, ATC reports:

```
ATC was unable to find a valid solution.
```

Angle-mode behavior matters for trigonometric solver examples. In degree mode, `solver(sin(x)-0.5)` is expected to return a degree-based solution.

22. Function study

Examples:

```
function study(x+1)
function study(x^2)
function study(x^2-4)
function study(_x^2)
function study(1/(x-1))
function study((x^2-1)/(x-1))
function study((1-x^2)/(x^2-4))
function study(x/(x^2-9))
function study((x+1)/(x^2+1))
```

Function study reports items such as domain, intercepts, asymptotes, symmetries, sign, monotony, concavity and codomain where supported.

23. Graph

Examples:

```
graph settings
graph(x)
graph(x;_2\2\1)
```

Graph behavior is console-based. Automated tests cover deterministic smoke paths and settings mutation; full visual inspection is still best done manually.

24. Guided calculation modules

ATC includes guided menus for:

```
triangles rectangles solver
arithmetic matrix solver
financial calculations
geometry calculations
statistics calculations
physics calculations
unit conversions
microeconomics calculations
```

These modules prompt for values and options. They are useful when the user prefers guided input over a single compact expression.

25. Financial, geometry, physics and unit conversion recipes

Finance:

```
financial calculations
100*15/100
```

Geometry:

```
geometry calculations
```

Physics:

```
physics calculations
```

Unit conversion:

```
unit conversions
```

Microeconomics:

```
microeconomics calculations
```

For guided menus, exact output depends on the selected options and entered values.

26. Time commands

Examples:

```
time
calendar(2026)
day of week(y2026m6d20)
day of week(d20m6y2026)
```

```
time difference calculations
actual time response
```

Interactive or continuously running tools include stopwatch, timer, big timer, clock and big clock variants. Use them manually when real time or opened windows are involved.

27. Sorting

Numeric sorting:

```
ascending order(3\1\2)
descending order(3\1\2)
ascending order(_3\1\_2\0)
descending order(_3\1\_2\0)
```

ASCII sorting:

```
ascii order
inverse ascii order
```

Some sorting commands can export reports.

28. TXT processing and command bridge

Core TXT commands:

```
predefine txt
solve txt
solve txt(name)
open txt("C:\Temp\file.txt")
to solve
enable txt detector
eliminate strings
atc from cmd
atc over cmd
auto solve txt
```

Typical flow:

1. Create a TXT file with one ATC command per line. 2. Run `predefine txt` and provide the path. 3. Run `solve txt`. 4. ATC creates an `_answers.txt` response file.

Example TXT input:

```
2+2
sqrt(9)
solver(x-2)
solve equation((x-1)(x-2))
simplify polynomial((x+1)(x-1))
2++
2+3
```

ATC should keep processing after an invalid line and report the syntax error for that line.

29. Scripting

The online guide documents scripting helpers such as:

```
print(...)
get(...)
sprint(...)
while
for
if
else
switch
case
cls()
break
return
replace
replace by index
count occurrences
delete x occurrences
is contained
calc
```

```
is equal
get positive value
is to write
is variable
is contained variable
is contained by index
strlen
```

Use scripting for repeatable command workflows. Keep scripts small and test them with known values before using them on larger calculations.

30. User functions

User functions live under the ATC user-functions folder. Autocomplete can suggest available user functions as `atc_<name>(` completions.

Use user functions when an expression should be reused by name.

31. Settings and environment commands

Common commands:

```
current settings
verbose resolution
verbose resolution(1)
verbose resolution(0)
numerical systems
si prefixes
actual time response
dimensions
window
about
auto adjust window
enable atc intro
disable atc intro
clean history
clean
exit
```

Use `current settings` to inspect persisted configuration.

32. File and folder commands

ATC manages folders such as:

- ATC data folder;
- Scripts examples;
- Source code;
- Strings;
- To solve;
- User functions.

Commands may open Explorer, Notepad or external viewers. In automated testing, these actions are mocked; in normal use they open real windows.

33. Export prompts

Some commands ask:

```
Export result? (Yes -> 1 \ No -> 0)
```

Use:

```
1
```

to export, or:

```
0
```

to skip export.

34. Verbose resolution

Enable:

```
verbose resolution(1)
```

Disable:

```
verbose resolution(0)
```

Verbose mode explains calculation processing. It is useful for learning and debugging, but normal calculations are quieter with verbose mode disabled.

35. Cookbook / Recipes

Online project documentation:

- ATC Cookbook

It contains practical workflows for scientific calculations, trigonometry, polynomial solving, matrices, statistics, DSP, TXT processing, verbose resolution and guided modules.

36. Recipes / Usage profiles

Engineering

```
solve equation(x^2-5*x+6)
create matrix(foo\2\2\3)
foo^2
unit conversions
physics calculations
```

Education

```
2+3*4
sin(pi/2)
mode
geometry calculations
function study(x^2)
```

DSP

```
sinc(0)
fft(1\0\0\0)
ifft(1\1\1\1)
sin(pi/2)+cos(0)
```

Statistics

```
avg(2\4\6)
min(3\_1\2)
max(3\_1\2)
statistics calculations
qfunc(0)
```

Finance

```
financial calculations
100*15/100
```

37. Best practices

Online project documentation:

- ATC Best Practices

Use it for reliable day-to-day workflows: start with small expressions, confirm angle mode, validate parentheses, choose the right solver path, use TXT files for repeatable batches and avoid undocumented behavior.

38. Practical troubleshooting

If an expression fails:

- check parentheses balance;
- check angle mode;
- check whether _ is needed for negative input;
- check whether the command is interactive;
- simplify the expression and test smaller pieces;
- use `verbose resolution(1)` for a more detailed trace;
- confirm whether the feature is documented and covered by regression tests.

39. Documentation and test coverage

Online project documentation:

- Testing

Historical validation recorded before the final 2.1.9 candidate:

These results do not approve the current candidate. Final validation must identify the source snapshot and executable hashes.

```
All: 376 passed, 0 failed
SolverComplex: 14 passed, 0 failed
Txt: 15 passed, 0 failed
Settings: 7 passed, 0 failed
```

This guide should evolve with the tests and the documented behavior of ATC 2.1.9.